COSC 369 - Software Engineering I

Project 1: Faculty Teaching & Office-Hours Schedule Generator

Design and Implementation Report

1. Project Overview

This project is a C program, **schedgen**, that reads two required input files — a syllabus / class-schedule file and a faculty / professor-information file — and produces, for every professor listed, a class-schedule table and a derived weekly office-hours schedule. The program is built as a set of reusable modules that are compiled into a static library, plus a small main program that links against that library. A Makefile drives the entire build: compiling the modules, archiving the static library, linking the executable, running it against the two input files, and invoking a Python script that turns the generated CSV data into a formatted Word document.

2. Module Organization

The source is split so that each file has one clear responsibility, and so that the parsing and scheduling logic can be reused (and unit-tested) independently of the program's entry point:

- **common.h / util.c** — shared constants, the `ResultCode` error-reporting type, and small time/string helpers (`time_to_minutes`, `minutes_to_ampm`, `trim`) used by every other module.
- **course.h / course.c** — the `Course` data model (one class meeting: code, title, lecturer, days, start/end time, room) and the loader that parses `class_schedule.txt`.
- **faculty.h / faculty.c** — the `Faculty` data model (a professor's id, display name, department, and office-hours policy) and the loader that parses `faculty_info.txt`.
- **scheduler.h / scheduler.c** — the scheduling logic. For one professor, it walks the parsed course list, builds a Monday-Friday table of class blocks, and applies the office-hours placement rule (section 5) to generate office-hour blocks for each day.
- **output.h / output.c** — rendering only. Turns a computed schedule into the plain-text CLASS SCHEDULE / OFFICE HOURS report and into CSV rows for the Python step. Contains no parsing or scheduling logic, so the on-screen/on-file format can change without touching the data model.
- **main.c** — the only file *not* included in the static library. It handles command-line arguments, opens the four files involved (two input, two output), and calls into the library modules in sequence. Keeping it separate is what makes the library/executable split (section 3) meaningful: main.c is the "program", the library is the "reusable engine".

This mirrors a standard layered design: data-model + parsing modules at the bottom, a scheduling/business-logic module in the middle, a rendering module above that, and a thin driver on top. Each module's header file is its contract; a module never reaches into another module's internals, only through the functions and structs its header declares.

3. Static Libraries in This Project

3.1 What a static library is

A static library is an archive of pre-compiled object files (`.o` files). On Linux/Unix, the conventional format is the `.a` ("archive") file created by the `ar` tool. Unlike a shared/dynamic library (`.so`), which is loaded at run time and can be shared in memory by several running programs, a static library's code is copied into the final executable at link time: whatever object code the linker needs to resolve undefined symbols is pulled out of the archive and embedded directly in the output binary. The result is a single self-contained executable with no run-time dependency on the archive file being present on the target machine.

3.2 How this project creates and links its static library

Every module except `main.c` (`util.c`, `course.c`, `faculty.c`, `scheduler.c`, `output.c`) is first compiled to an object file with `gcc -c`, exactly as any C source file would be. Those five object files are then archived into `lib/libschedgen.a` with:

```
ar rcs lib/libschedgen.a obj/util.o obj/course.o obj/faculty.o \
                         obj/scheduler.o obj/output.o
```

The `r` flag inserts (or replaces) the given object files in the archive, `c` creates the archive if it does not already exist, and `s` writes an index of the symbols each object file defines so the linker can search the archive efficiently instead of scanning every member. `main.c` is compiled separately into `obj/main.o`, and the executable is produced by linking that one object file against the archive:

```
gcc -std=c11 -Wall -Wextra -Iinclude -g obj/main.o -Llib -lschedgen -o bin/schedgen
```

`-Llib` tells the linker to look for libraries in the `lib/` directory, and `-lschedgen` tells it to link against `liba/libschedgen.a` (`-l<name>` maps to `lib<name>.a`). The linker resolves every symbol `main.o` references — `course_list_load`, `faculty_list_load`, `scheduler_build`, `output_write_report`, `output_write_csv` — by pulling the corresponding object files out of the archive and embedding them in `bin/schedgen`.

3.3 Why a static library here

The assignment specifically asks for a static library to demonstrate organizing a project into modules with a library boundary. A static library is also the natural choice for a single-machine command-line tool like this one: there is only one executable that ever uses these modules, the program is meant to be copied to a lab machine and run without an installation step, and there is no benefit here from the memory sharing or independent upgrade path that a shared library would offer to multiple, separately-installed programs.

4. How the Makefile Manages Dependencies and Automation

The Makefile expresses the whole build as a dependency graph of targets, each with a rule for how to produce it from the files it depends on:

- **Pattern rule for object files** — `$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c include/*.h` tells `make` that any `obj/X.o` depends on `src/X.c` and on every header file. If a header changes, every `.o` file is considered out of date and is recompiled the next time `make` runs; if only one `.c` file changes, `make` recompiles only that one object file, because it compares each target's modification time against its prerequisites' modification times and skips a rule whose target is already newer than everything it depends on.
- **Static-library target** — `$(STATIC_LIB): $(LIB_OBJECTS)` means the archive is only rebuilt when at least one of the five library object files is newer than the archive itself, which in turn only happens when the corresponding source or a header changed.
- **Executable target** — `$(EXECUTABLE): $(MAIN_OBJECT) $(STATIC_LIB)` re-links the final binary whenever either `main.o` or the archive itself is newer than the existing executable.
- **Automation targets** — `run` depends on `all`, so typing `make run` first brings the executable up to date (rebuilding only what changed) and only then executes it against the two files in `data/`. The same target then invokes `make report`, which calls `python3 scripts/make_docx.py` to turn the freshly generated CSV into a Word document. This is the "Python program invoked from the Makefile" the assignment allows for turning the C program's output into a Word or Excel document: the C program owns the data (parsing and scheduling), and the Python script owns only the final document layout.
- **Housekeeping targets** — `clean` removes `obj/`, `lib/`, and `bin/` so the next build starts from source; `distclean` additionally removes the generated report files, giving a fully reproducible build from a clean checkout.

In short, the Makefile's job is to never do more work than necessary (thanks to timestamp-based dependency checking) while still guaranteeing that a single command, `make run`, takes the project from source files all the way to a finished Word document.

5. The Office-Hours Placement Rule

The two input files supply everything the program needs except one thing: a rule for turning a teaching schedule into an office-hours schedule. Rather than hard-coding one rule for all professors, each professor's `faculty_info.txt` row supplies three policy values — this is the "office-hours table provided by the lecturer" the assignment refers to:

- **daytime_block_end_min** — how late a daytime office-hour block runs (e.g. 1:00 PM).
- **evening_block_len_min** — how long an office-hour block placed before an evening class should be (e.g. 120 minutes).

- **evening_cutoff_min** — the clock time at or after which a class is treated as an "evening" class rather than a daytime one (e.g. 4:00 PM).

For each weekday and each professor, `scheduler_build` (in `scheduler.c`) separates that day's classes into daytime and evening meetings using `evening_cutoff_min`, then:

- if the professor has a daytime meeting that day, one office-hour block is placed from the end of the *last-ending* daytime meeting until `daytime_block_end_min`;
- if the professor has an evening meeting that day, a second office-hour block of length `evening_block_len_min` is placed immediately before the *earliest-starting* evening meeting;
- a day with neither kind of meeting gets no office-hour block at all;
- every generated block is clamped to an 8:00 AM-6:00 PM campus-hours window so the rule can never place office hours outside reasonable working hours, even with an unusual class time.

Applied to the sample professor in the assignment (Monday/Wednesday COSC 330 at 9:30-10:45 AM, Tuesday/Thursday COSC 369 at 9:30-10:45 AM, and a Wednesday evening COSC 631 at 5:00-7:30 PM, with `daytime_block_end_min` = 1:00 PM, `evening_block_len_min` = 120, `evening_cutoff_min` = 4:00 PM), this rule reproduces the exact office-hours table given in the assignment: 10:45 AM-1:00 PM on Monday, Tuesday, Wednesday and Thursday, plus a second 3:00-5:00 PM block on Wednesday, and nothing on Friday.

6. Input File Location: Local Files, Not the Internet

Both required input files are read from the local filesystem, passed to the program as command-line arguments (defaulted to `data/class_schedule.txt` and `data/faculty_info.txt` by the Makefile's `run` target). The program opens them with ordinary C standard-library file I/O (`fopen`/`fgets`/`fclose` in `course.c` and `faculty.c`); there is no networking code anywhere in the project.

This design was chosen deliberately, for several reasons:

- **Scope and reliability.** The assignment's own description treats the two files as artifacts a department already has — a syllabus/class-schedule table and a faculty roster — not as data that needs to be fetched from a remote service. Reading local files means the program has no dependency on network availability, no need for HTTP client code or a JSON/XML parser, and nothing that can fail because a remote server is down or requires authentication.
- **Reproducibility for grading.** A lab machine may not have outbound internet access at all, and even if it does, fetching "the class schedule" from the internet would require deciding which of countless possible remote sources counts as authoritative. Local files make the exact same run reproducible on any machine that has the project directory, which matters for a course project that will be compiled and run by someone other than its author.
- **Separation of concerns.** If the schedule data did need to come from a web service in some other deployment, the right place for that is a small preprocessing step (or a different data-acquisition

module entirely) that writes the same pipe-delimited local file format the parser already expects — not a change to the parsing or scheduling modules themselves. Keeping `course_list_load` and `faculty_list_load` as pure "read this local file" functions keeps that door open without adding any actual networking to this project.

7. Error Handling

Every module function that can fail returns a `ResultCode` (`RC_OK`, `RC_ERR_FILE_OPEN`, `RC_ERR_PARSE`, `RC_ERR_CAPACITY`, `RC_ERR_ARGS`) instead of a bare integer, so a call site can tell exactly what went wrong. A malformed line in either input file is reported to `stderr` with its line number and contents and is then skipped, so one bad line in a large file does not abort the entire run; a file that cannot be opened at all, or a fixed-size array that would overflow (`MAX_COURSES`, `MAX_FACULTY`, `MAX_DAILY_BLOCKS`), stops that step and is reported clearly rather than silently truncating data.

8. Sample Output

See `output/report.txt`, `output/report.csv`, and `output/report.docx` in the submission for the program's output on the sample input files in `data/`. The plain-text report for the first sample professor reproduces the exact CLASS SCHEDULE and OFFICE HOURS tables given as the sample output in the assignment.